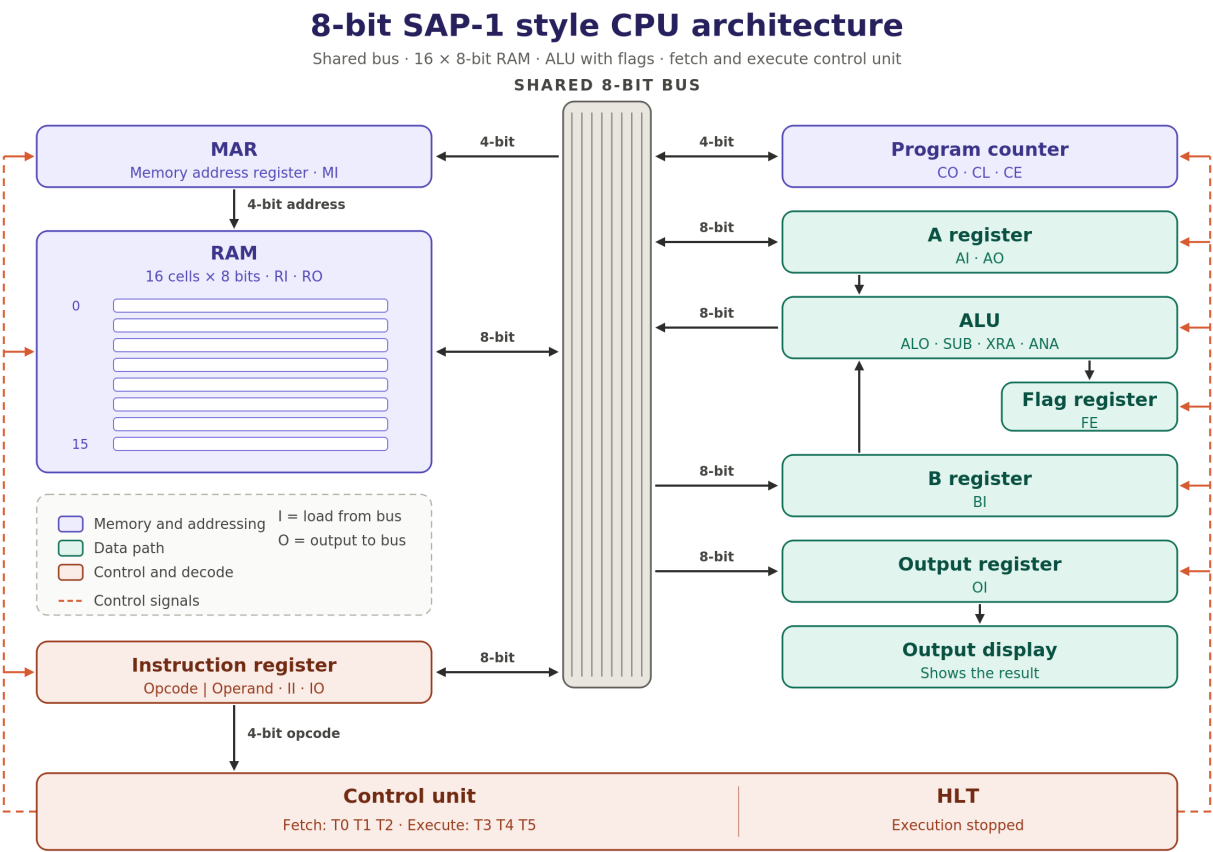


8-bit SAP-1 style CPU

Architecture block diagram and role of each block



Role of each block

Program counter (PC) Holds the address of the next instruction. Counts up (CE) after every fetch.	Memory address register (MAR) Holds the RAM address being accessed, loaded from the bus (MI).	RAM Stores the program and data: 16 cells of 8 bits. Read (RO) or write (RI).
Instruction register (IR) Holds the current instruction. Opcode goes to the control unit, operand goes to the bus.	Control unit (CU) Generates every control signal in steps T0-T5: fetch, then execute. HLT stops the CPU.	Shared bus One 8-bit path for all transfers. Only one block drives it at a time; others read.
A register (accumulator) Holds one ALU input and stores the result of each operation.	B register Holds the second ALU input, loaded from the bus (BI).	ALU Does the arithmetic and logic: add, subtract, XOR, AND.
Flag register Keeps 2 status bits: Zero (bit 1) and Carry (bit 0), updated by FE.	Output register Holds the final result to be shown, loaded from the bus (OI).	Output display Shows the output register value to the user.

Instruction set (ISA)

11 instructions. Every instruction is one byte: a 4-bit opcode and a 4-bit operand (a RAM address 0 to 15).

Bit 7	6	5	4	3	2	1	Bit 0
OPCODE IR[7:4]				OPERAND IR[3:0]			

Opcode goes to the control unit. Operand goes to the bus (IO) and is used as a RAM address or jump target. NOT, HALT and OUT ignore the operand.

Mnemonic	Opcode	Operand	Operation	ALU select	Flags
LOAD (LDA)	0000	addr	$A = M[\text{addr}]$	-	-
STORE (STA)	0001	addr	$M[\text{addr}] = A$	-	-
ADD	0010	addr	$A = A + M[\text{addr}]$	0000	Z, C
SUB	0011	addr	$A = A - M[\text{addr}]$	0001	Z, C
AND	0100	addr	$A = A \text{ AND } M[\text{addr}]$	0100	Z
OR	0101	addr	$A = A \text{ OR } M[\text{addr}]$	0101	Z
JUMP	0110	addr	$PC = \text{addr}$	-	-
HALT	0111	not used	Stop execution (HLT)	-	-
OUT	1000	not used	Output register = A	-	-
XOR	1001	addr	$A = A \text{ XOR } M[\text{addr}]$	0110	Z
NOT	1010	not used	$A = \text{NOT } A$	0111	Z

Colours: purple = memory, teal = ALU, red = control flow, amber = output. **ALU select** is the 4-bit code {ALO, SUB, XRA, ANA} given to the ALU. **Flags:** FLAG[1] = Zero (Z), FLAG[0] = Carry (C), updated when FE is active. A dash means no ALU operation or no flag update.

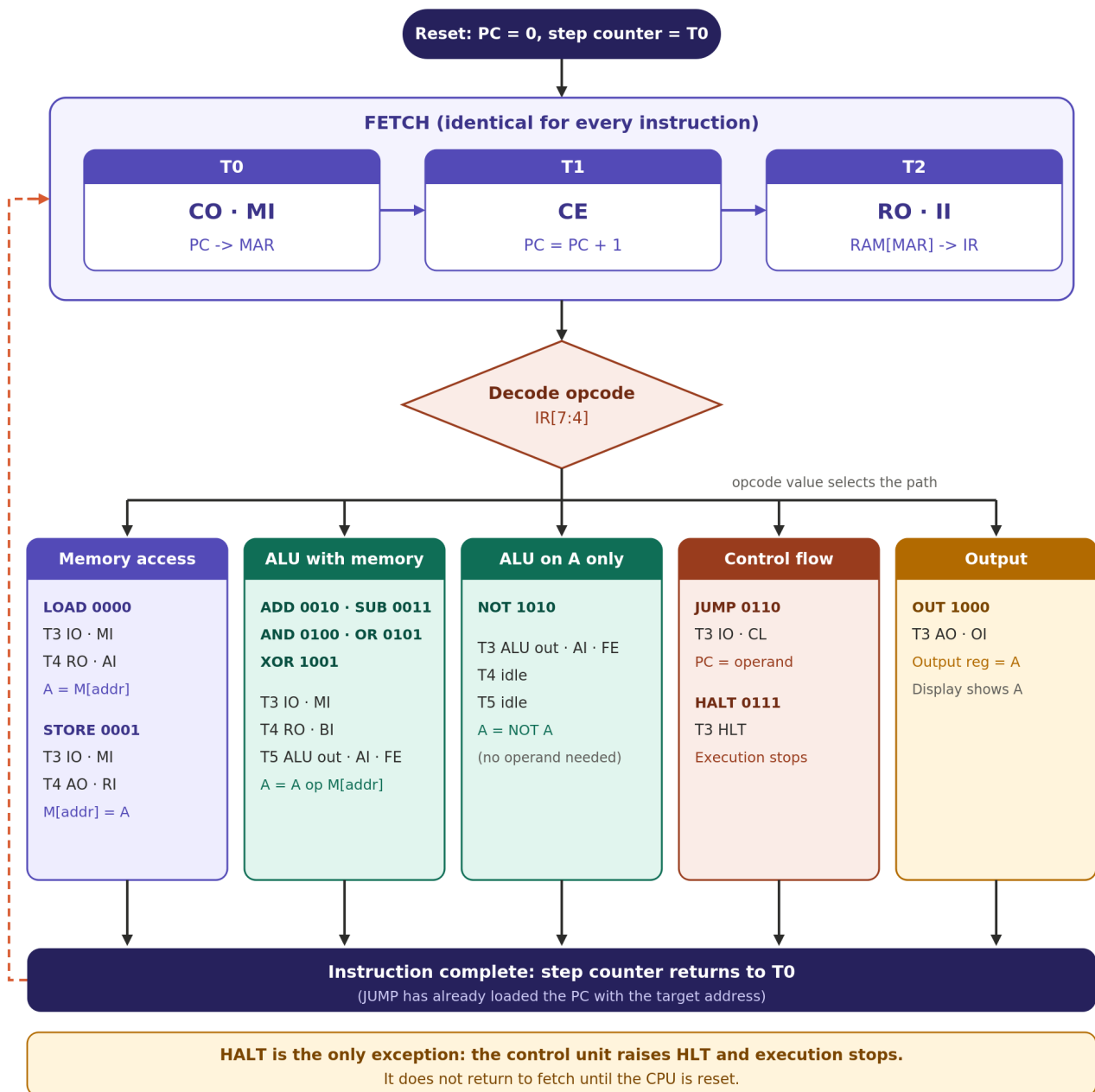
Instruction cycle

Every instruction takes six clock steps. T0 to T2 fetch it (identical for all instructions). T3 to T5 execute it.

Step	Signals	What happens
T0	C0, MI	Program counter puts its address on the bus, MAR loads it
T1	CE	Program counter counts up to the next address
T2	R0, II	RAM sends the instruction, instruction register loads it

ISA execution flow

From reset, through fetch and decode, to the execute path of each instruction.



Execute-phase control signals (T3 to T5)

Use this as the truth table for the control matrix. Fetch (T0 to T2) is the same for every instruction.

Instr	Opcode	T3	T4	T5	Effect
LOAD	0000	IO, MI	RO, AI	idle	Address to MAR, RAM data into A
STORE	0001	IO, MI	A0, RI	idle	Address to MAR, A written to RAM
ADD	0010	IO, MI	RO, BI	ALU out, AI, FE	Operand to B, sum into A, flags set
SUB	0011	IO, MI	RO, BI	ALU out, AI, FE	Operand to B, difference into A
AND	0100	IO, MI	RO, BI	ALU out, AI, FE	Bitwise AND into A
OR	0101	IO, MI	RO, BI	ALU out, AI, FE	Bitwise OR into A
XOR	1001	IO, MI	RO, BI	ALU out, AI, FE	Bitwise XOR into A
NOT	1010	ALU out, AI, FE	idle	idle	Invert A, no memory access
JUMP	0110	IO, CL	idle	idle	Program counter loaded with operand
HALT	0111	HLT	idle	idle	Execution stops
OUT	1000	A0, OI	idle	idle	A copied to output register

Signal key: MI = MAR in, RI/RO = RAM in/out, II/O = instruction register in/out, AI/AO = A register in/out, BI = B register in, OI = output register in, FE = flag enable, CO = counter out, CE = counter enable (count up), CL = counter load (jump), HLT = halt. "ALU out" puts the ALU result on the bus using the ALU select code from the ISA table.